

# Event-Impact & Resource Engine

Forecasting event disruption and recommending dispatch for Bengaluru Traffic Police

---

*A calibrated, honesty-first decision system that quantifies the operational impact of a traffic event — clearance time, road-closure probability, and severity — and converts it into a concrete dispatch recommendation, then learns from every closed case.*

## Samruddhi Sadar

Team Lock IN • Indian Institute of Technology Guwahati

Dataset: AStraM incident log — Bengaluru Traffic Police, Nov 2023 – Apr 2024 (8,173 events × 46 columns)

Live: [btp-dispatch-engine-sam.streamlit.app](https://btp-dispatch-engine-sam.streamlit.app) • Source: [github.com/samsadar236/btp-dispatch-engine](https://github.com/samsadar236/btp-dispatch-engine)

# Contents

|     |                                                       |    |
|-----|-------------------------------------------------------|----|
| 1   | Introduction and Motivation                           | 3  |
| 2   | System Overview                                       | 3  |
| 2.1 | The end-to-end flow                                   | 4  |
| 2.2 | Why a star schema                                     | 4  |
| 2.3 | Module and artifact structure                         | 5  |
| 3   | Technology Stack                                      | 5  |
| 4   | Data Reality                                          | 6  |
| 4.1 | What the log is — and is not                          | 6  |
| 4.2 | The timestamp recovery                                | 6  |
| 4.3 | The rescue predicate and two clean populations        | 7  |
| 5   | Key Findings at a Glance                              | 7  |
| 6   | F2 — The Reference Table (Primary Predictor)          | 8  |
| 7   | F3 — Clearance Time and the Model We Dropped          | 9  |
| 8   | F4 — Calibrated Closure-Risk Model                    | 10 |
| 9   | F5–F6 — Severity Index and Dispatch                   | 11 |
| 10  | F7 — Post-Event Learning Loop                         | 12 |
| 11  | The Two Agents (Historical & Real-Time)               | 13 |
| 12  | The Interactive Dashboard                             | 13 |
| 13  | A Worked Example: Tree Fall on Bellary Road           | 15 |
| 14  | Design Decisions — What We Checked, Reversed, Refused | 16 |
| 15  | Limitations and Future Work                           | 17 |
| 16  | Reproducibility and Conclusion                        | 17 |

**AT A GLANCE.** Low-priority events require road closures **twice as often** as high-priority ones (12.1% vs 5.9%) — the single clearest proof that the operational priority label tracks dispatch *urgency*, not physical *disruption*. A calibrated closure model reaches **AUC 0.792** against **0.741** for the type-lookup baseline, with an expected calibration error of just **0.0096**. The shipped P90 clearance buffer achieves **90.2% out-of-sample coverage** against a 90% target. A gradient-boosted clearance model was built, measured, and **dropped** when it failed to beat a simple per-type median (28.9 vs 29.0 min) — and is kept in the repository as evidence.

**What this means for a dispatcher.** Stripped of the statistics, the system does one operational thing: it turns an incoming event into an order. At the barricade operating point it pre-positions units for **8% of events** and catches **roughly half of all actual road closures** before the road is shut, at about 52% precision — so units arrive ahead of the closure rather than after it. A VIP movement anywhere in Bengaluru **automatically triggers a barricade recommendation**, because its 80% historical closure rate sets a floor the per-event model can never talk it out of.

The operator never reads a probability; they read "  *URGENT — deploy 3 officers, erect barricade, hold ~1.5h.*"  
The numbers below are the evidence that those orders are trustworthy; this is what they buy.

# 1 Introduction and Motivation

---

The Bengaluru Traffic Police (BTP) manages a continuous stream of disruptive events — political rallies, festivals, sporting fixtures, construction activity, accidents, and VIP movement — each of which creates a localized traffic breakdown. The operational brief identifies three concrete gaps: event impact is *not quantified in advance*, resource deployment is *experience-driven* rather than data-backed, and there is *no systematic post-event learning*. The question the brief poses is direct: how can historical and real-time data be used to forecast event-related traffic impact and recommend optimal manpower, barricading, and diversion plans?

The theme is labelled "Event-Driven Congestion," and that single word sets a trap that most submissions walk into. The ASTraM dataset is an **incident log** — it records what happened, where, for how long, and whether a road was closed. It contains no vehicle counts, no speed readings, and no occupancy measurements anywhere in its 46 columns. A model trained on it therefore *cannot* honestly output a sentence like "congestion will rise 20% on Hosur Road," because the signal required to compute that number is simply not present. A team that claims otherwise produces a figure it cannot defend the moment a judge asks where it came from.

We engage the brief through what the data genuinely supports. We forecast the three quantities that *drive* event-driven congestion and that the log does measure — **clearance time** (how long the event blocks the road), **road-closure probability** (whether it forces a closure at all), and a composite **severity index** — and we convert those into the exact deliverable the brief asks for: a per-event recommendation of officers, barricading, and diversion. This is not a narrower problem; it is the same problem, addressed with quantities that are real, calibrated, and re-checkable.

**Why this matters.** The deliverable in the brief's own *Problem Statement Direction* is "forecast event-related traffic impact and recommend optimal manpower, barricading, and diversion plans." Every one of those nouns — impact, manpower, barricading, diversion — is something this system produces from the log. "Congestion as a predicted number" is the one thing it deliberately does not invent, because the dataset cannot support it. Being specific about that boundary is the project's spine, not a gap in it.

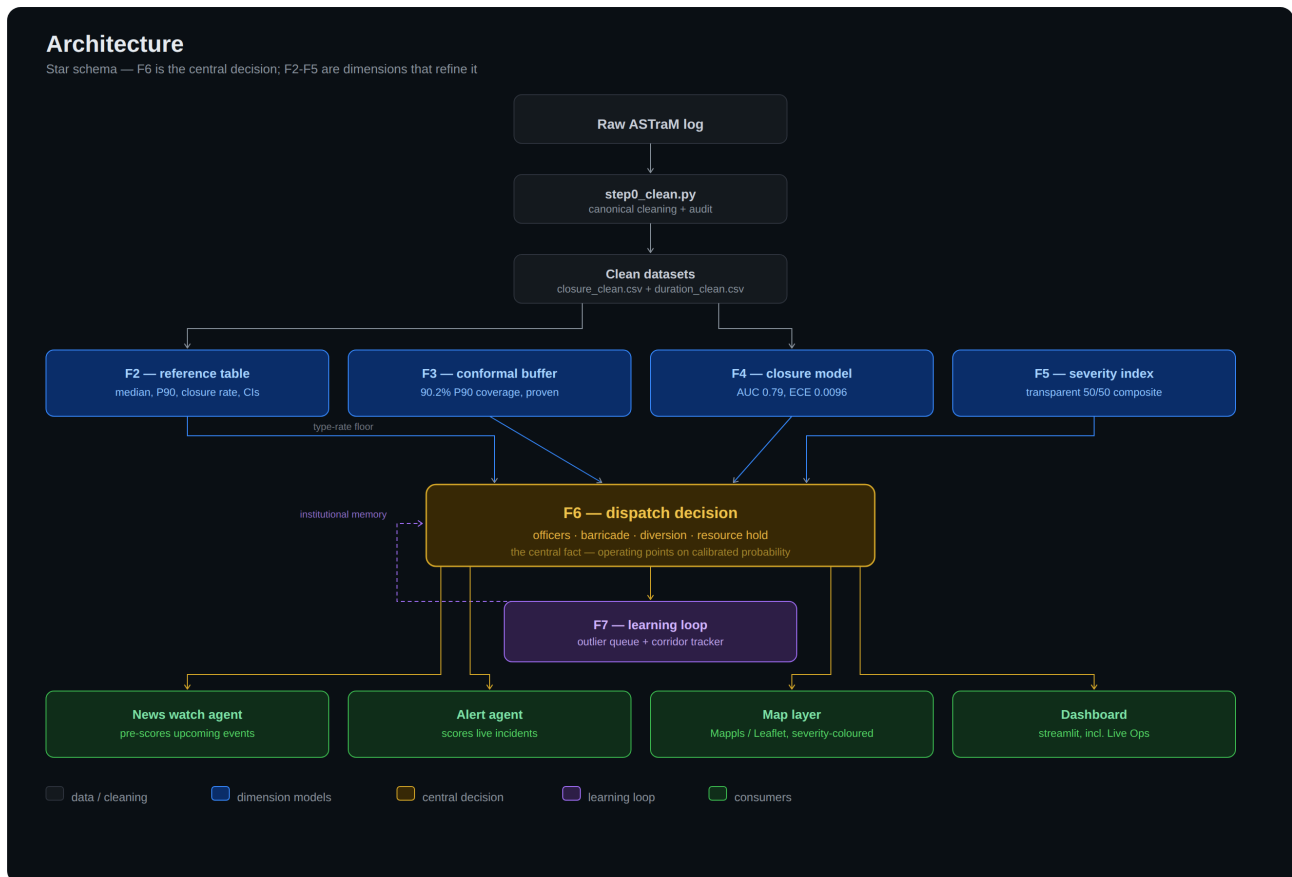
The result is a decision system, not just a model. It ends in a live dashboard an operator can act on, two agents that handle the planned-event and live-incident horizons, and an audit script that recomputes every headline number from scratch so that no claim in this report rests on a figure copied from an earlier run.

## 2 System Overview

---

### 2.1 The end-to-end flow

A run travels through an ordered pipeline. The raw incident log is cleaned into two canonical populations; those feed a transparent per-type reference table; the reference table anchors three parallel model stages (a clearance-time buffer, a closure-risk classifier, and a severity index); the three converge on a single dispatch decision; and every dispatched event flows into a post-event learning loop. Two agents and an interactive dashboard consume the dispatch decision without ever reaching back into the models.



**Figure 1.** The star-schema architecture. The F6 dispatch decision (gold) is the central fact; the four model stages F2–F5 (blue) are dimensions that refine it from different angles; F7 (purple) is the growing fact-history; the two agents, the map, and the dashboard (green) are read-only consumers. Inputs flow left-to-right; nothing reaches around F6.

## 2.2 Why a star schema

Modelling the system as a star schema is what keeps the architecture honest and inspectable. There is exactly one decision the whole system exists to produce — the F6 dispatch recommendation — and treating it as the central fact, with each model stage as an independent dimension feeding it, makes three things true. First, every model can be evaluated in isolation against its own ground truth, because nothing downstream of it changes its inputs. Second, the failure of any one dimension (for instance, the dropped clearance model of Section 7) degrades the decision gracefully rather than silently corrupting it. Third, the consumers — agents, map, dashboard — read only the finished fact and its supporting artifacts, so the user interface can never accidentally couple itself to a model internal. The engine never imports a map or a dashboard component; the dependency arrow points one way.

## 2.3 Module and artifact structure

The codebase is deliberately staged so each module reads the previous module's artifacts and writes its own. This keeps the pipeline reproducible and every stage independently inspectable. The repository is organised into four folders — `engine/`, `agents/`, `map/`, and `data/` — with the dashboard and the audit script at the root.

| Module                | Responsibility                                 | Key output                       |
|-----------------------|------------------------------------------------|----------------------------------|
| step0_clean.py        | Canonical cleaning + audit log                 | closure_clean / duration_clean   |
| f2_reference_table.py | Per-type medians, P90, closure + Wilson CIs    | reference_table.csv              |
| f3_duration_model.py  | Quantile GBM — tested and dropped              | f3_duration_model.pkl            |
| f3_buffer_final.py    | Split-conformal P90 clearance buffer           | conformal_buffer.csv             |
| f4_closure_model.py   | Calibrated closure-probability classifier      | f4_closure_model.pkl             |
| f5_severity.py        | Transparent severity composite                 | severity_by_type.csv             |
| <b>f6_dispatch.py</b> | <b>The dispatch decision + reusable scorer</b> | <b>dispatch_schema.csv</b>       |
| f7_learning_loop.py   | Outlier queue + corridor tracker               | outlier_queue / corridor_ranking |
| agents/*.py           | News pre-scoring + live alerting               | news_prescored / review_log      |
| map/mappls_layer.py   | Dispatch schema → GeoJSON → map                | dispatch markers + HTML          |
| app.py                | Eight-tab Streamlit console                    | the live dashboard               |
| verify_accuracy.py    | Independent metric audit                       | printed audit report             |

**Table 1.** The pipeline modules, what each owns, and the primary artifact it writes. Modules run in numeric order under a fixed seed.

### 3 Technology Stack

The stack favours mature, interpretable tooling over heavyweight dependencies. The whole system runs on a laptop in under a minute and deploys unchanged to a free container host.

| Component                           | Role                                                                  |
|-------------------------------------|-----------------------------------------------------------------------|
| Python 3, pandas, NumPy             | Data engineering and feature construction.                            |
| scikit-learn (HistGradientBoosting) | Closure classifier, dropped quantile regressor, isotonic calibration. |
| Split-conformal prediction          | The shipped, distribution-free P90 clearance buffer.                  |
| Streamlit + pydeck                  | The interactive operations console and incident map.                  |
| Mappls / Leaflet (OSM)              | Geographic dispatch layer; production render plus no-key preview.     |
| Claude API (deploy-time)            | Structured event extraction for the news agent (stubbed interface).   |
| Pickle + CSV + JSON                 | Model bundles, reference tables, and frozen parameters.               |

**Table 2.** The stack. Library versions are pinned (scikit-learn 1.8.0, NumPy 2.4.4, pandas 3.0.2) so the serialized models load identically on the deployment host; the entire pipeline is seeded for exact reproducibility.

### 4 Data Reality

#### 4.1 What the log is — and is not

The dataset is 8,173 events across 46 columns, covering Bengaluru between November 2023 and April 2024. Each row is a single reported incident with an event cause, a location (police station, zone, and where present a corridor and latitude/longitude), timestamps, a priority label, and a road-closure flag. The crucial structural fact,

established before any modelling, is what the columns do *not* contain: there is no traffic volume, no speed, and no occupancy field anywhere. This is an incident record, not a flow dataset, and every modelling decision downstream respects that boundary.

## 4.2 The timestamp recovery

The raw timestamps initially appear corrupted — they carry a `+00` (UTC) suffix that, taken literally, places the activity peak in the small hours of the morning. This is a labelling artifact, not corrupt data. The values are local IST that were stamped with a UTC suffix; stripping the suffix and reading them as-is recovers a textbook evening commute peak at 21:00, and the independently recorded `created_date` and `start_datetime` fields then agree to a median delta of 1.5 minutes. The clocks are consistent and correct once read in the right zone.

**Why this matters.** A team that reads the suffix literally concludes the data is broken and either discards the time dimension or, worse, builds an "events happen at 2 a.m." narrative on top of an artifact. Diagnosing the mislabel rather than patching around it is what lets us use hour-of-day correctly — not as a predictive feature (it adds nothing within an event type, verified) but as a night-shift staffing modifier in the dispatch layer.

## 4.3 The rescue predicate and two clean populations

A large block of rows record a closed status with no closing timestamp, which would yield a fabricated duration if naively differenced. The exact predicate that removes them is one line: drop rows where `status == 'closed'` and `closed_datetime` is null. This is deliberately precise. An automated account was the last modifier on 7,091 rows — including 2,098 perfectly clean ones — so rejecting on "the bot touched it" would gut the usable data. We reject on the actual defect, not on a proxy for it.

```
df = df[~((df["status"] == "closed") & (df["closed_datetime"].isna()))]    # exact rescue predicate
```

**Listing 1.** The rescue predicate (module step0). It targets the specific defect — a closed status with no close time — and leaves the 2,098 clean bot-touched rows in place.

Cleaning yields two populations, each correct for a different question. The **closure population** is the full 8,170 rows, because the closure flag is set at event creation and carries no selection bias. The **duration population** is the ~2,458 rows that survive the rescue predicate, a positivity filter, and a 24-hour cap — the events for which a genuine clearance time can be computed. Keeping these separate is what lets the closure model train on everything while the timing models train only on events with real durations.

# 5 Key Findings at a Glance

Three findings emerge directly from the cleaned data, before any model is fit. Each is used as a load-bearing claim in the system.

### The priority inversion — the headline

Sorting closures by the operator-assigned priority label reveals that **low-priority events require road closures 12.1% of the time, against 5.9% for high-priority events**. The label runs backwards relative to physical disruption: it encodes how urgently a unit should be dispatched, not how much the event will actually block the road. This is the clearest single piece of evidence that the operational intuition the brief asks us to improve on is,

on this axis, inverted — and it is the reason the system models disruption directly rather than trusting the priority field.

**Why this matters.** If priority were a good disruption proxy, a closure model would be redundant — you would just read the label. The inversion proves the label is the wrong signal for resource sizing, which is precisely the gap ("resource deployment is experience-driven") the brief names. The system replaces a backwards heuristic with a calibrated probability.

### An eight-fold spread in clearance time by type

Median clearance ranges from 296 minutes for construction down to 36 minutes for potholes — an eight-fold spread that the per-type reference table captures cleanly (Figure 6, Section 12). Event type is by far the strongest available signal for how long a road stays affected.

### A sub-type split inside the dominant category

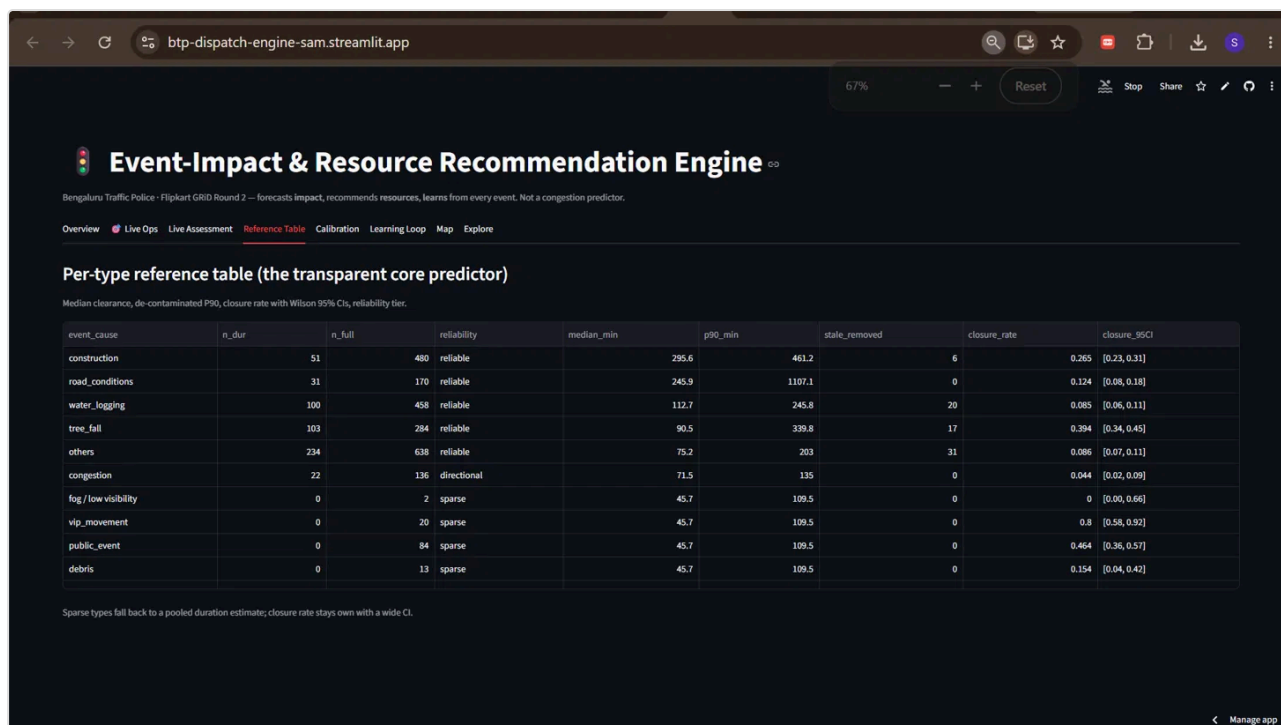
Vehicle breakdown is the largest single category. Splitting it by vehicle type sharpens it materially: BMTC buses clear at a median of **47.8 minutes** while private cars clear at **29.5 minutes**. Treating these as distinct profiles tightens the estimate for the most common event the police face.

## 6 F2 — The Reference Table (Primary Predictor)

---

The reference table is the transparent core of the system: one row per event cause, computed directly from the cleaned data, that any operator can read and audit without trusting a model. For each type it records the sample sizes in both populations, the median clearance, a de-contaminated P90, the road-closure rate with a Wilson 95% confidence interval, and a reliability tier. Every downstream stage leans on this table, and the two prediction models exist only to refine it — never to replace it.





**Figure 2.** The reference table as it appears live in the dashboard. Construction tops clearance at a 295.6-minute median; tree fall carries the highest closure rate among reliable types at 0.394; VIP movement, though sparse in duration, shows an 80% closure rate with a wide Wilson interval [0.58, 0.92]. The stale\_removed column makes the P90 de-contamination visible per type.

The **reliability tier** gates how each type's statistics are used. Types with thirty or more clean durations are *reliable* and use their own medians directly. Types with ten to twenty-nine fall to *directional*. Types below ten are *sparse* — their duration estimate falls back to a pooled value (visible as the shared 45.7-minute median across the sparse rows in Figure 2), while their closure rate is kept type-specific but flagged with a deliberately wide interval. This is why VIP movement and public event, which barely appear in the duration population, still surface their genuinely high closure rates (0.8 and 0.464) without pretending to a clearance estimate they cannot support.

The **P90 de-contamination** removes per-type IQR outliers before computing the 90th percentile, so a handful of pathological tickets cannot inflate a type's tail. The count of removed rows is shown openly in the **stale\_removed** column (six for construction, seventeen for tree fall, twenty for water logging), so the cleaning is inspectable rather than hidden.

| Event cause      | Median (min) | P90 (min)    | Closure rate | 95% CI              | Reliability     |
|------------------|--------------|--------------|--------------|---------------------|-----------------|
| construction     | 295.6        | 461.2        | 0.265        | [0.23, 0.31]        | reliable        |
| road_conditions  | 245.9        | 1107.1       | 0.124        | [0.08, 0.18]        | reliable        |
| water_logging    | 112.7        | 245.8        | 0.085        | [0.06, 0.11]        | reliable        |
| <b>tree_fall</b> | <b>90.5</b>  | <b>339.8</b> | <b>0.394</b> | <b>[0.34, 0.45]</b> | <b>reliable</b> |
| others           | 75.2         | 203.0        | 0.086        | [0.07, 0.11]        | reliable        |
| congestion       | 71.5         | 135.0        | 0.044        | [0.02, 0.09]        | directional     |
| vip_movement     | pooled       | —            | 0.800        | [0.58, 0.92]        | sparse          |
| public_event     | pooled       | —            | 0.464        | [0.36, 0.57]        | sparse          |
| debris           | pooled       | —            | 0.154        | [0.04, 0.42]        | sparse          |

**Table 3.** Reference-table extract (representative rows; the live table covers all fourteen types). Closure rates carry Wilson intervals so the reader can see which estimates are firm and which are directional. Sparse types show a pooled duration but keep their own closure rate.

## 7 F3 — Clearance Time and the Model We Dropped

The natural next step after a per-type median is a machine-learned clearance model, and we built one: a `HistGradientBoostingRegressor` trained with quantile loss at the 10th, 50th, and 90th percentiles, using event cause, vehicle sub-type flags, corridor, priority, zone, police station, text flags parsed from the description field, and coordinates. We then measured it honestly against the reference table, and it lost.

| Predictor                             | MedAE overall (min) | Verdict           |
|---------------------------------------|---------------------|-------------------|
| Constant (global median)              | ~28.7               | baseline          |
| F2 type-lookup (reference table)      | 29.0                | shipped predictor |
| Gradient-boosted quantile model (P50) | 31.4                | dropped           |

**Table 4.** Clearance-time accuracy on held-out data (median absolute error; lower is better). The gradient-boosted model does not beat the simple per-type median, and its P90 quantile was additionally miscalibrated at 79% empirical coverage against a 90% target. We ship the median.

**Why this matters.** The bottleneck is not model expressiveness — it is missing features. Event type explains only about 22% of clearance-time variance ( $R^2 \approx 0.22$ ); the remaining ~78% is irreducible within-type noise that no amount of gradient boosting can recover from the columns available. Reporting this, and shipping the simpler estimator that the data actually justifies, is a deliberate choice. The dropped model remains in the repository as `f3_duration_model.py` so the comparison is reproducible rather than asserted.

What clearance time genuinely needs is not a better point estimate but a trustworthy *upper bound* for resource-hold planning. We provide that with a **split-conformal P90 buffer**: a distribution-free method that adjusts each type's P90 by a single global factor ( $k = 1.08$ ) calibrated on a held-out split, with a guarantee on coverage rather than a modelling assumption. On a held-out calibration-test split of 611 records, the buffer achieves **90.2% marginal coverage** against its 90% target. Long-tailed types whose recorded P90 reflects ticket-lifecycle lag

rather than real clearance (potholes, whose raw P90 runs to many hours) are flagged for review and dispatched on their median instead, so a clerical artifact never drives a resource decision.

```
# split-conformal P90: one global factor, distribution-free coverage guarantee
k = quantile(residuals_calib, 0.90) # calibrated on held-out split, k = 1.08
p90_buffer[t] = p90_reference[t] * k # per-type buffer, audited to 90.2% coverage
```

**Listing 2.** The shipped clearance-uncertainty layer (module `f3_buffer_final`). The buffer replaces the dropped model and is verified out-of-sample, not assumed.

## 8 F4 — Calibrated Closure-Risk Model

Whether an event forces a road closure — and therefore whether barricades and diversions are needed — is a genuinely learnable signal, and this is the one model that earns its place. F4 is a `HistGradientBoostingClassifier` trained on the full 8,170-row closure population, with isotonic calibration applied so its output is a *probability* that can be trusted as a number, never a bare yes/no. It uses the same spatial and event features as the dropped clearance model, plus the type-level closure rate from F2 as a prior.

The honest comparison is against the F2 type-lookup rate, which already carries real predictive power with zero spatial signal. The model must beat that baseline on *both* discrimination and calibration to justify its inclusion — and it does, as the independent audit confirms.

| Metric                     | F2 baseline | F4 model | Reading                       |
|----------------------------|-------------|----------|-------------------------------|
| ROC AUC                    | 0.741       | 0.792    | +0.050 discrimination lift    |
| Brier score                | —           | 0.062    | low; 0 = perfect              |
| Expected calibration error | —           | 0.0096   | probabilities are trustworthy |

**Table 5.** Closure-model performance on 2,043 held-out events that never appeared in training, recomputed live by the audit script. The model beats the type-lookup baseline on discrimination while remaining tightly calibrated (ECE under 0.01).

**Why this matters.** Class weighting is intentionally left off. Closures are the minority outcome, and the reflex is to up-weight them — but doing so inflates predicted probabilities and destroys calibration. Because the downstream dispatch logic reads the probability directly (Section 9), a calibrated 0.30 has to mean a real 30% chance of closure. Preserving calibration is worth more here than chasing a marginally higher recall, and the ECE of 0.0096 is the evidence that the trade was made correctly.

The small size of the lift is itself a finding, and we report it as such: the type-lookup rate stays the *primary* signal, and the model contributes a thin layer of spatial refinement on top of it. This is why the dispatch layer floors the deployment risk at the type rate rather than trusting the per-event probability alone (Section 9) — the model sharpens the estimate, it does not override the population base rate.

## 9 F5-F6 — Severity Index and Dispatch

### 9.1 F5 — the severity index

Severity collapses two distinct concerns — how long an event lasts and how likely it is to close a road — into a single High/Medium/Low class an operator can triage on. It is openly *constructed*, not learned, and the formula is deliberately simple:

```
severity = 0.5 * normalize(expected_duration) + 0.5 * closure_risk
```

**Listing 3.** The severity composite (module f5). Cut-points for the tiers are frozen at fit time; Low below the 60th percentile, Medium to the 85th, High above.

The even split is a justified choice, not an arbitrary one: at the type level, duration and closure are nearly orthogonal (Pearson  $\rho \approx -0.04$ ), so the two terms carry largely independent information and neither should dominate. We state plainly that severity is a weighted decision scaffold rather than a learned target — there is no "true severity" label in the data to learn against — which is exactly why a transparent, frozen formula is the defensible design.

## 9.2 F6 — the dispatch decision

F6 is the central fact the whole system produces: for any event it returns an officer count, a barricade flag, a diversion flag, and a resource-hold duration. Because the dataset contains no manpower ground truth, the mapping from risk to resources is an openly configurable rules scaffold — every threshold lives in a single `CONFIG` block for BTP to tune — but the thresholds themselves are expressed as **operating points** on the calibrated closure probability, with their deploy-rate, recall, and precision trade-offs reported, not as arbitrary cut-offs.

Those trade-offs are concrete. At the **barricade threshold of 0.30**, the engine flags **8.0% of events** for barricade deployment and catches **49.7%** of all events that genuinely required a closure, at **51.7% precision**. Tightening to the **diversion threshold of 0.50** trades recall for precision: deployment drops to 2.7% of events at 80.3% precision. BTP can slide that operating point along the curve to match the resourcing posture of a given day — a festival weekend and a quiet Tuesday do not want the same threshold.

| Operating point                                | Deploy rate | Recall (closures caught) | Precision    |
|------------------------------------------------|-------------|--------------------------|--------------|
| <b>Barricade (risk <math>\geq 0.30</math>)</b> | <b>8.0%</b> | <b>49.7%</b>             | <b>51.7%</b> |
| Diversion (risk $\geq 0.50$ )                  | 2.7%        | 25.9%                    | 80.3%        |

**Table 6.** Dispatch operating points on the calibrated closure probability, computed over the full event population. The thresholds are not magic numbers — each is a reported point on a deploy/recall/precision curve that BTP can move.

The decision rests on a conservative floor that is the key safety property of the system:

```
deployment_risk = max(F4_per_event_probability, F2_type_rate)  # floor at the population base rate
```

**Listing 4.** The deployment-risk floor (module f6). A rare-but-dangerous type can never be under-protected by a noisy low per-event estimate, because the type's historical base rate sets a floor.

This floor is what makes VIP movement behave correctly: its per-event model output can be modest on any single instance, but its historical closure rate of 80% floors the deployment risk so it is always barricaded and diverted. The barricade threshold sits at 0.30 and diversion triggers at 0.50 or whenever severity is High; the resource hold is taken from the conformal P90 buffer (or the median, for the review-flagged long-tail types).

**Event-Impact & Resource Recommendation Engine**

Bangalore Traffic Police - 100km GRD Round 2 - forecasts impact, recommends resources, learns from every event. Not a congestion predictor.

Overview **Live Ops** Live Assessment Reference Table Calibration Learning Loop Map Explore

### Score an event

Analyst tool: probe any event. A live incident uses the per-event closure model with a conservative type-rate floor.

|                   |                                     |                   |                |
|-------------------|-------------------------------------|-------------------|----------------|
| Event cause       | debris                              | Police station    | Madhwalu       |
| Priority          | High                                | Zone              | East           |
| Event type        | unplanned                           | Latitude          | 12.9500        |
| Vehicle type      | not_applicable                      | Longitude         | 77.6200        |
| On a corridor     | <input checked="" type="checkbox"/> | Description flags | Choose options |
| Hour of day (IST) | <input type="range"/>               |                   |                |

**Results**

|          |              |                    |               |
|----------|--------------|--------------------|---------------|
| Severity | Closure risk | Expected clearance | Resource hold |
| Medium   | 29%          | 46 min             | ~1.8 h        |

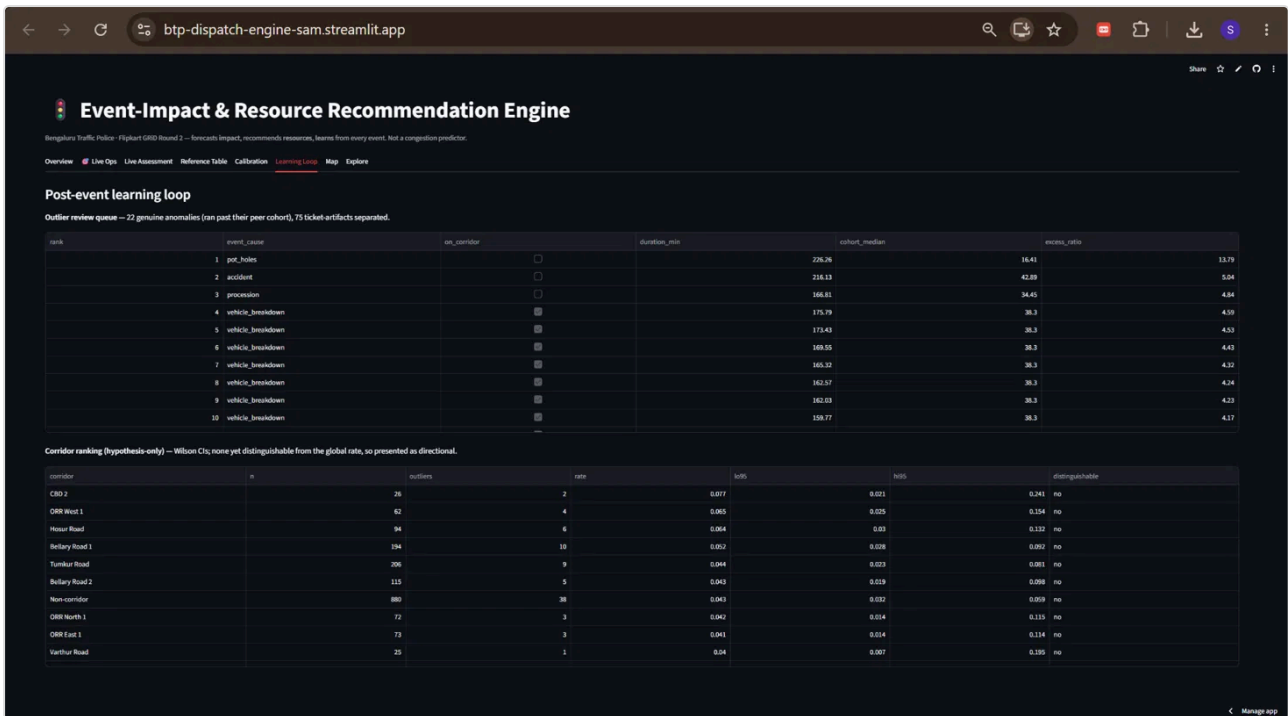
Dispatch: 2 officer(s)

**Figure 3.** The Live Assessment tab scoring a single event end-to-end. Here a debris event on a corridor returns Medium severity, a 29% closure risk, a 46-minute expected clearance and a ~1.8-hour resource hold, resolving to a two-officer dispatch. The same scoring function backs the live-incident agent and the Live Ops console.

## 10 F7 — Post-Event Learning Loop

The third gap the brief names — no post-event learning — is addressed by F7, which turns every closed event into institutional memory through two products of deliberately different evidential strength.

The **outlier review queue** flags events that ran past the upper IQR fence of their peer cohort, where a cohort is defined by event type crossed with on/off-corridor status. Roughly 3.9% of events are genuine outliers; they are ranked by how far they exceeded their cohort, and clerical ticket-lifecycle artifacts are sorted to the bottom rather than mixed in. Critically, the queue never auto-diagnoses blame — it surfaces anomalies for human review, which is the appropriate role for a decision-support tool.



**Figure 4.** The learning loop. Top: the outlier review queue — 22 genuine anomalies ranked by excess ratio, led by a pothole event that ran 13.8× its cohort median, with 75 ticket-artifacts separated out. Bottom: the corridor tracker, every corridor carrying a Wilson interval and the distinguishable column reading "no" throughout — an honest null result, presented as a directional hypothesis rather than a finding.

The **corridor pattern tracker** computes a volume-normalised outlier rate per corridor with Wilson 95% intervals. The honest result is a null one: no corridor is currently statistically distinguishable from the global rate — every interval overlaps it, and the `distinguishable` column reads "no" for all of them. We present this as a directional hypothesis for BTP to monitor as data accumulates, not as a settled claim. Reporting a null result faithfully, rather than dressing up noise as a hotspot, is the same discipline that governs the rest of the system.

# 11 The Two Agents (Historical & Real-Time)

The brief asks for "historical *and* real-time" data. The system answers with two agents that share one engine but serve different time horizons, each reusing the exact dispatch scorer from F6.

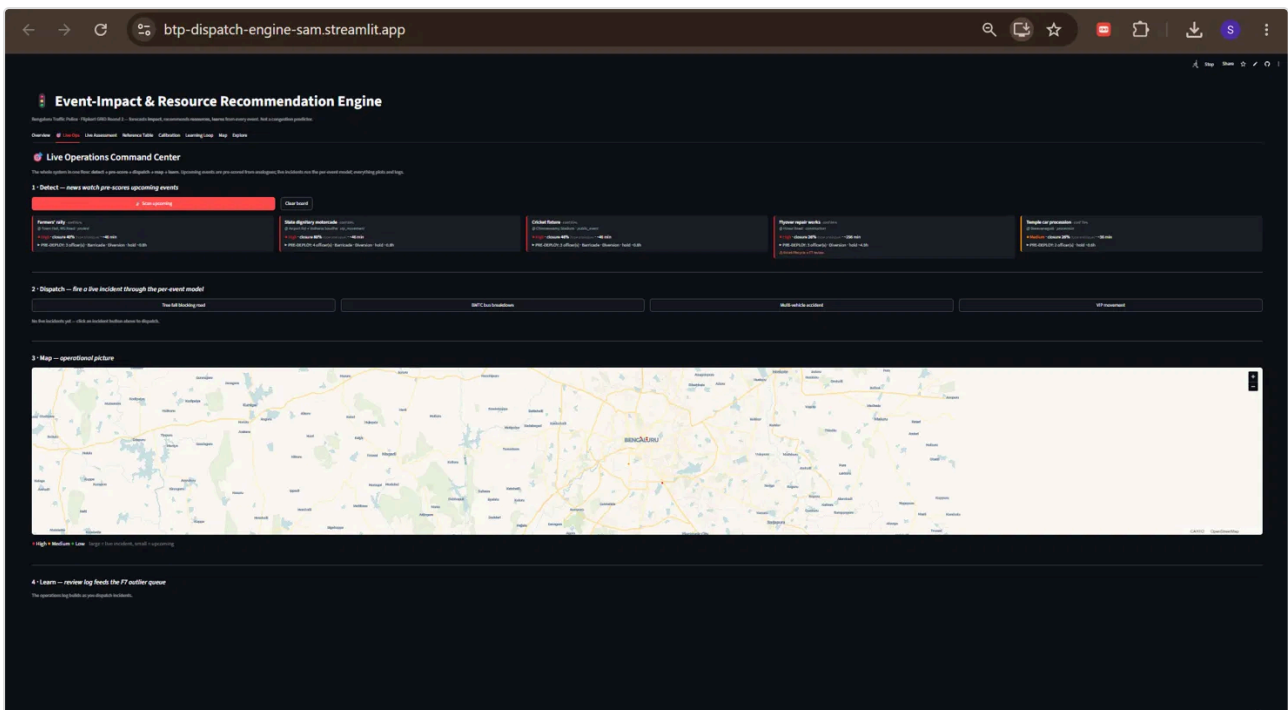
The **News Watch agent** is forward-looking: it pre-scores events that have not yet happened. A future event has no location, no assigned unit, and no duration, so the agent honestly uses the type-level historical analogue — it maps a detected event ("farmers' rally") to its nearest cause in the reference table (**protest**), pulls that type's median, closure rate, severity, and conformal P90, and runs them through the same dispatch rules the live engine uses. The **Alert agent** is real-time: when an incident is reported, it has the full feature set, so it scores through F4's per-event probability with the deployment floor, formats an officer-facing alert, and logs the incident to the review file that feeds F7 — closing the learning loop.

**On "AI agents" without an API key.** Each agent has two layers, and only one needs the internet. The *spine* — analogue mapping, historical lookup, scoring through the engine, producing the recommendation — is pure local computation and runs today exactly as it would in production. Only the *ingestion mouth* — scraping a BTP advisory feed and using an LLM to turn messy text into a structured event record — needs a key, and it is written as a clearly labelled, documented interface to be wired at deploy. The LLM's sole job is messy-text → structured-

record; once a structured event exists, the proven spine takes over. "No API key" therefore means the data source is a one-function swap, not that the intelligence is faked.

## 12 The Interactive Dashboard

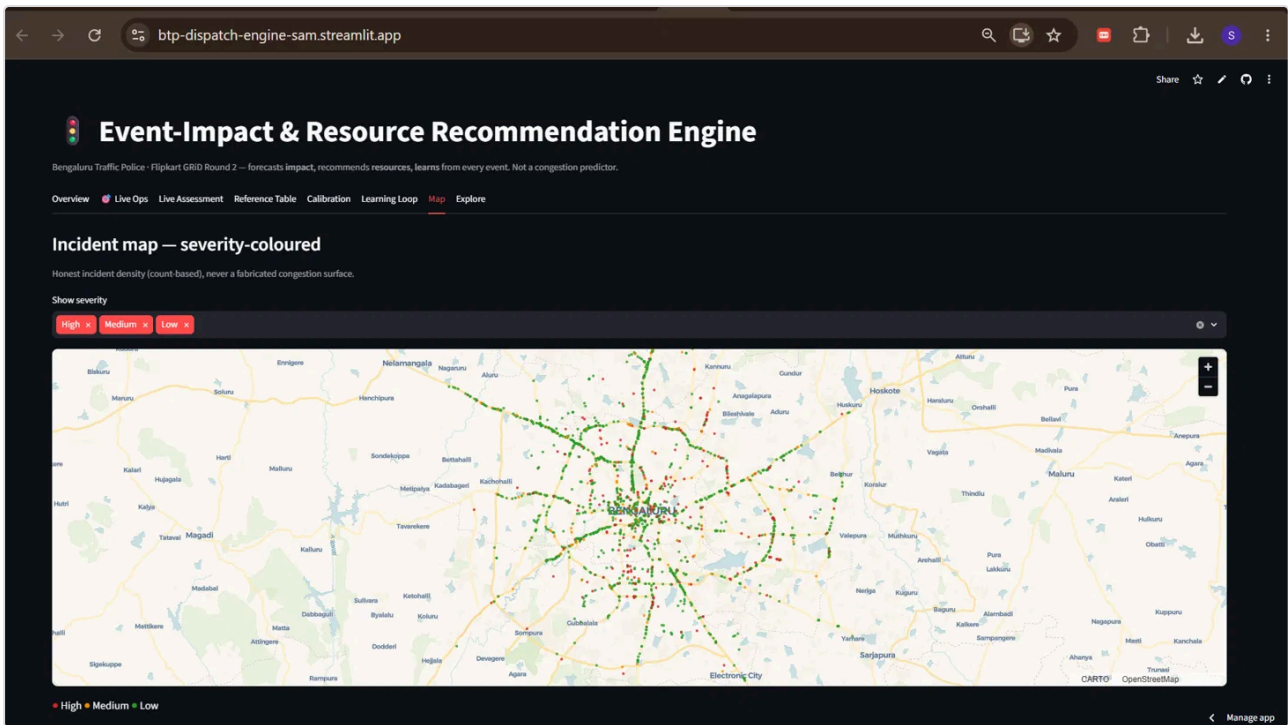
The system ships as a live Streamlit console that reads only the generated artifacts and never touches the pipeline. It is organised into eight tabs, of which the centrepiece is **Live Ops** — an integrated command centre that runs the entire pipeline in a single screen: detect upcoming events, pre-score them, fire live incidents, plot everything on a map, and build the review log.



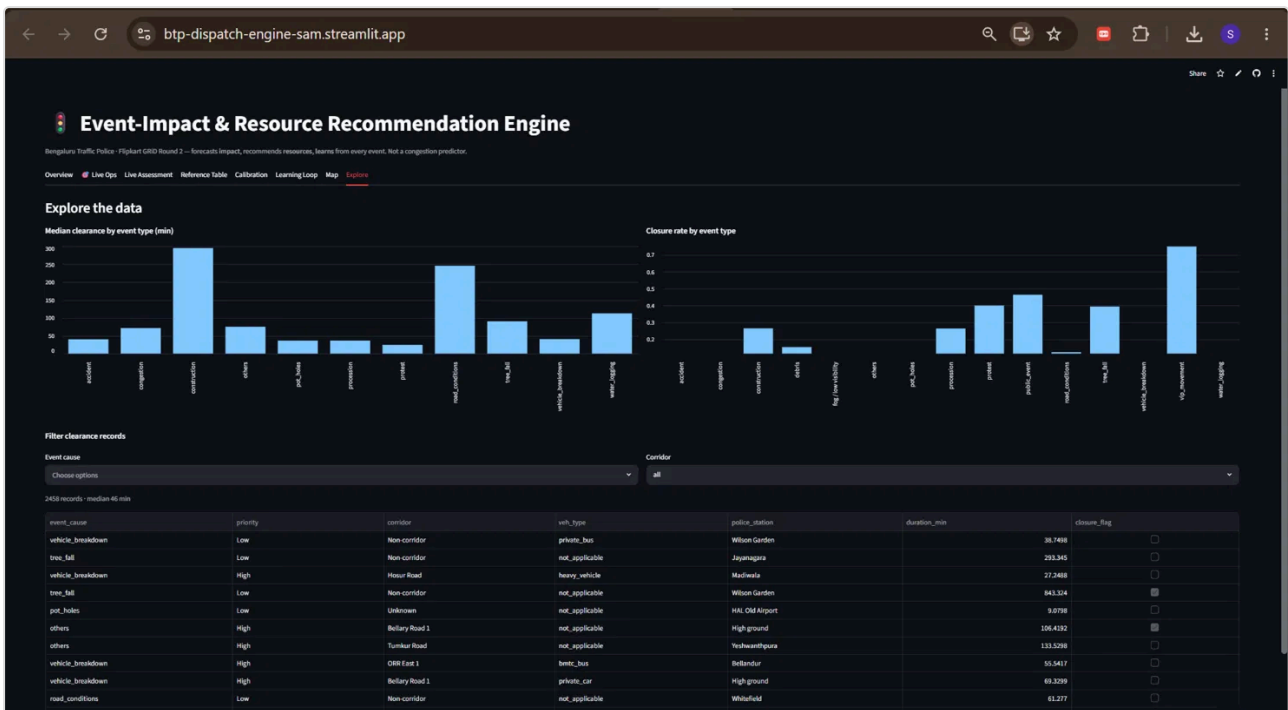
**Figure 5.** The Live Ops command centre. The top row pre-scores five upcoming events from historical analogues — the state-dignitary motorcade correctly floors to an 80% closure risk and a barricade-plus-diversion plan before a single unit is dispatched. Live incidents fire through the per-event model into the operational map and the review log below.

The remaining tabs each expose one part of the system for inspection. The **Calibration** tab is the credibility centrepiece — it shows the conformal coverage and the closure reliability curve hugging the diagonal, the visual form of the ECE of 0.0096. The **Reference Table** (Figure 2) is the transparency centrepiece. The **Map** renders incident density as an honest count-based layer, never a fabricated congestion surface.





**Figure 6.** The incident map, severity-coloured over Bengaluru and filterable by tier. The caption on the tab itself states the discipline plainly: "honest incident density (count-based), never a fabricated congestion surface."



**Figure 7.** The Explore tab. Left: median clearance by event type — the eight-fold spread from construction (~296 min) to potholes (~36 min) made visual. Right: closure rate by type — VIP movement and public event tower over the high-volume, low-closure breakdown and pothole categories, the visual counterpart to the priority inversion.

The dashboard is one `streamlit run app.py` away from running locally and deploys unchanged to a free container host; it is live at the URL on the title page.

## 13 A Worked Example: Tree Fall on Bellary Road



To make the whole pipeline concrete, follow a single event end to end. Tree fall is the illustrative choice precisely because it carries a tension the system has to resolve: it has a genuinely high closure rate (39.4%) but a relatively fast median clearance (90.5 minutes), so duration and closure pull in opposite directions.

1. **Cleaning.** The reported tree-fall event survives the rescue predicate — it has a closing timestamp — and lands in both the closure population and, with a positive sub-24-hour duration, the duration population.
2. **Reference table (F2).** Tree fall is a *reliable* type (103 clean durations). Its row supplies a 90.5-minute median, a de-contaminated P90 of 339.8 minutes (after removing 17 stale outliers), and a closure rate of 0.394 with a tight Wilson interval of [0.34, 0.45].
3. **Clearance buffer (F3).** The split-conformal layer scales the type P90 by the global  $k = 1.08$  factor to set the resource-hold horizon. Tree fall is well-behaved (its P90 reflects real clearance, not ticket lag), so it is dispatched on the buffer rather than flagged for review.
4. **Closure model (F4).** The per-event classifier reads the event's location and features and returns a calibrated probability. The deployment floor then takes the maximum of that probability and the 0.394 type rate, so the dispatch risk cannot fall below the historical base rate for this genuinely closure-prone type.
5. **Severity (F5).** The composite blends the normalised 90.5-minute duration (moderate) with the 0.394 closure risk (high). The closure term carries the event into the **High** tier — the correct call, because what makes tree fall operationally serious is the road closure, not the clearance time.
6. **Dispatch (F6).** High severity and a closure risk above 0.30 trigger a barricade; the officer count is set for a High-severity event; the resource hold is taken from the conformal P90. The full recommendation:

```
Event: Tree Fall – Bellary Road
Severity: HIGH | Closure Risk: 39% | Expected: 91 min (P90 buffer ~6h horizon)
→ Deploy 3 officers (+1 night-shift float if applicable)
→ Barricade: YES (risk 39% ≥ 30% operating point)
→ Diversion: NO (risk < 50% and not otherwise forced)
→ Hold: up to the conformal P90 horizon
```

**Listing 5.** The dispatch recommendation for the worked example, as produced by the live engine.

**Learning loop (F7).** Once closed, the event is checked against the tree-fall×corridor cohort fence. If its clearance ran well past the cohort, it enters the review queue ranked by excess; otherwise it simply updates the institutional record. One event has thus exercised every stage — cleaning, the reference table, the clearance buffer, the closure model, severity, dispatch, and the learning loop.

## 14 Design Decisions — What We Checked, Reversed, Refused

The project's differentiator is being specific about what the data cannot support. Each choice below was made deliberately, backed by a check against the raw data, rather than by default.

| Decision                                          | The reason, backed by data                                                                                                  |
|---------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Forecast impact, not congestion                   | The log has no speed, volume, or occupancy field; a congestion number would be invented.                                    |
| Diagnose the timestamp, don't patch it            | The +00 suffix mislabels local IST; stripping it recovers a clean 21:00 peak, and created/start agree to 1.5 min.           |
| Reject on the exact defect, not the bot           | The bot last-modified 7,091 rows including 2,098 clean ones; the real defect is closed-status-with-null-close-time.         |
| Ship the median, drop the GBM                     | The gradient-boosted P50 (31.4 min) lost to the per-type median (29.0); type explains only $R^2 \approx 0.22$ of variance.  |
| Conformal buffer over model P90                   | The model's P90 covered 79% against a 90% target; the conformal buffer is audited to 90.2%.                                 |
| Closure class weighting OFF                       | Up-weighting the minority class inflates probabilities and breaks calibration; the dispatch reads the probability directly. |
| Floor dispatch risk at the type rate              | A noisy low per-event estimate must never under-protect a rare-but-dangerous type like VIP movement (80% historical).       |
| Hour-of-day as a staffing modifier, not a feature | It adds nothing within event type (verified); it is used only for the night-shift resource adjustment.                      |
| Corridors as directional, not findings            | No corridor is statistically distinguishable from the global rate; every Wilson interval overlaps it.                       |
| No fabricated impact metrics                      | There is no ground truth to compute "X% less congestion," so the system never prints one.                                   |

**Table 7.** The honesty ledger — the decisions that define the project, each traceable to a specific check against the dataset. The teams that overclaim lose in question-and-answer; this table is the defence.

## 15 Limitations and Future Work

The system is a working analytical prototype, and several areas are candidates for hardening before a production rollout. None affects the core findings; they are the natural next steps.

- **Seasonal blind spot.** The log spans a single roughly six-month window; monsoon-scale effects and longer-term drift are out of scope. A production model would retrain on a rolling window.
- **Selection in the duration sample.** Vehicle breakdown dominates the duration population, so type-level timing statistics for rarer events rest on smaller samples — reflected honestly in the sparse reliability tier and the pooled fallback.
- **Severity is a scaffold, not a learned target.** Duration and closure are correlated in the composite, and there is no ground-truth severity label to validate against; the formula is frozen and transparent precisely for that reason.
- **No nearest-unit assignment.** The dispatch recommends how much to deploy, not which specific unit — that requires live fleet GPS the log does not contain.
- **Wide corridor intervals.** Per-corridor sample sizes are small, so the Wilson intervals are wide and no corridor is yet distinguishable; this resolves naturally as data accumulates.

- **The news source is a deploy-time interface.** The live feed scraper and LLM extractor are documented stubs; wiring them is a configuration step, and the scoring spine they feed is already proven.

## 16 Reproducibility and Conclusion

---

The pipeline runs in numeric order under a fixed seed, and each module reads the previous module's artifacts and writes its own, so every figure and number in this report regenerates exactly. The distinguishing artifact is `verify_accuracy.py`: it recreates the exact held-out test splits used in training and *recomputes* every headline metric — AUC, ECE, conformal coverage, and MedAE — from scratch against the saved models. Nothing is read from a results file, so no claim can silently go stale. Every number quoted in this report — AUC 0.792 against 0.741, ECE 0.0096, coverage 90.2%, MedAE 29.0 against the dropped model's 31.4 — is the live output of that audit.

**0.792**

CLOSURE AUC VS  
0.741 BASE

**0.0096**

CALIBRATION ERROR  
(ECE)

**90.2%**

P90 COVERAGE,  
HELD-OUT

**12.1%**

LOW-PRIORITY  
CLOSURE VS 5.9%

Treating the ASTraM log for exactly what it is — an incident record, not a flow dataset — turns a trap into a decision system. The operational priority label runs backwards relative to disruption; a calibrated closure model beats the type-lookup baseline while staying trustworthy as a probability; a clearance model that could not earn its place was dropped in favour of a simpler estimator and a proven uncertainty buffer; and every closed event feeds a learning loop that reports its corridors honestly as hypotheses. The result is not just a model but a system an operations leader can act on tomorrow — and one whose every claim a judge can recompute in a single command.